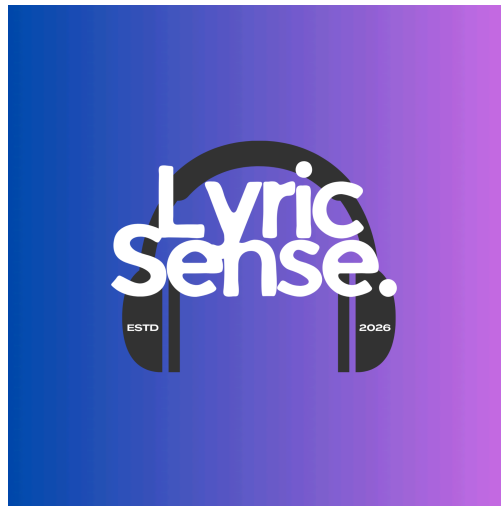


Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)



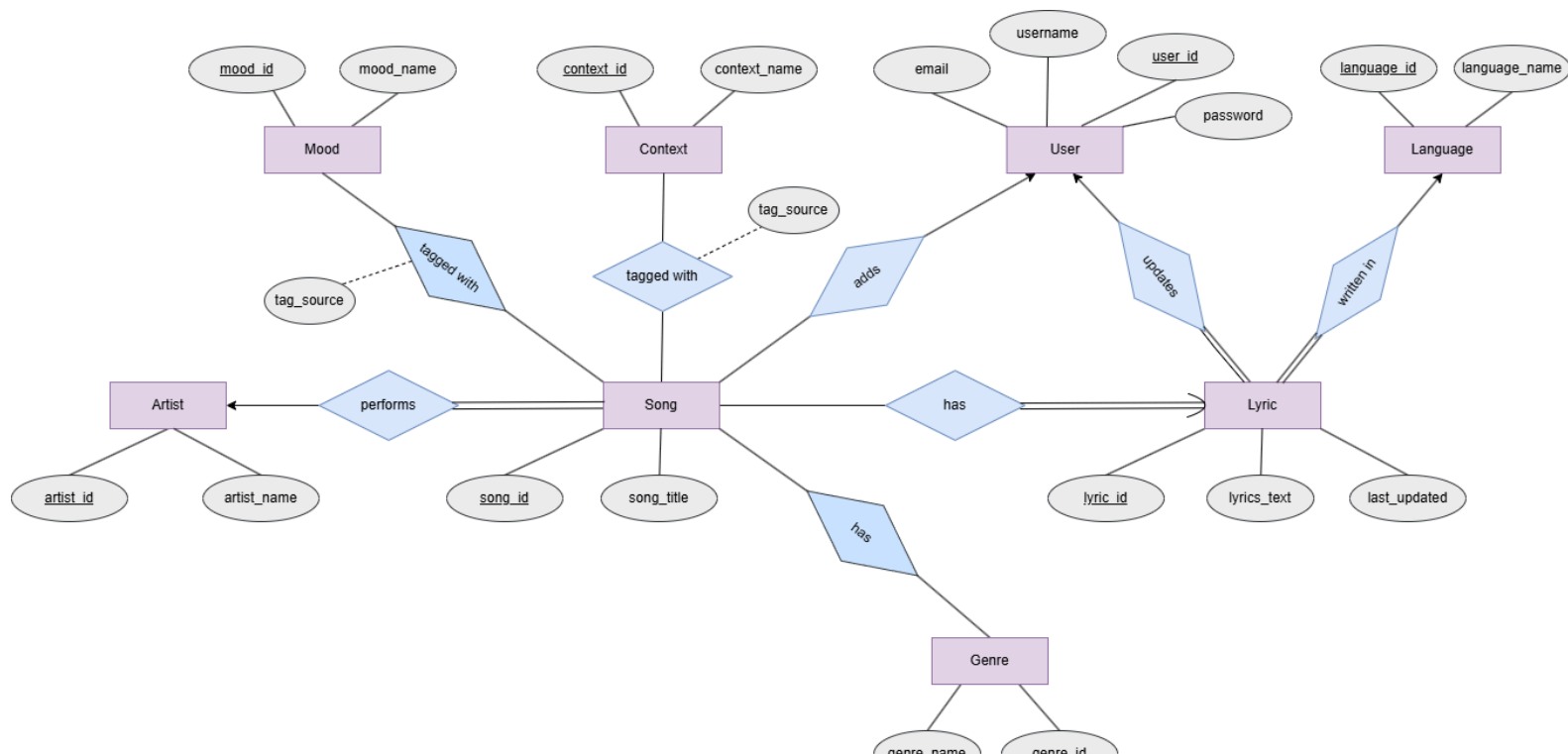
Lyric Sense: The Right Song for Right Now

Final Deliverable: Brief Report

[Link to Project Files](#)

1. Database Design

1.1 E-R Diagram



Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

1.2 Schema Statements

Artists(artist_id, artist_name)

Genres(genre_id, genre_name)

Languages(language_id, language_name)

Songs(song_id, song_title, artist_id, genre_id)

- FK: artist_id references Artists(artist_id)
- FK: genre_id references Genres(genre_id)

Lyrics(lyric_id, song_id, language_id, lyrics_text, last_updated)

- FK: song_id references Songs(song_id)
- FK: language_id references Languages(language_id)

Moods(mood_id, mood_name)

Contexts(context_id, context_name)

Users(user_id, email, username, password)

TaggedWithMoods(song_id, mood_id, tag_source)

- FK: song_id references Songs(song_id)
- FK: mood_id references Moods(mood_id)

TaggedWithContexts(song_id, context_id, tag_source)

- FK: song_id references Songs(song_id)
- FK: context_id references Contexts(context_id)

2. Database Programming

2.1 Database Hosting

The LyricSense database is primarily hosted on the University of Virginia Computer Science MySQL server and accessed through phpMyAdmin. The database is stored on this centralized server rather than on individual local machines, allowing multiple users to connect and interact with the same dataset.

Users running the LyricSense application locally connect to the UVA MySQL server over the network using PHP. The database management system supports concurrent access, enabling multiple users to query and modify data simultaneously without interfering with one another.

Access to the UVA database is restricted. Users must have valid database credentials and be connected to the UVA network, either through the Cisco Secure Client VPN or by being physically on the UVA network. Without VPN access or proper authentication, the database cannot be reached.

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

Due to storage and upload limitations on the UVA CS server, the full database may be too large for all users to import through phpMyAdmin. In such cases, users may work with a reduced dataset or rely on a shared instance of the database maintained by the project owner. This ensures that the application remains usable even when full database replication is not feasible.

Although the database is currently hosted on the UVA server for development purposes, the system is portable and can be deployed on other environments such as local MySQL (XAMPP) or cloud platforms like Google Cloud SQL by modifying the database connection settings.

2.2 Application Hosting

The LyricSense application is a PHP based web application that runs locally using PHP's built in development server. The application is not deployed on a public production server, but instead runs on a user's local machine and communicates with a remote or local database.

To run the application, the following environment is required:

- **PHP version 8.x or later**
- **A terminal or command line interface**
- **One of the following database environments:**
 - **UVA CS MySQL server (requires Cisco Secure Client VPN)**
 - **Local MySQL server (via XAMPP)**
 - **Cloud hosted MySQL instance (e.g., Google Cloud SQL)**

2.3 Deployment Instructions

Option A: UVA CS Server (Primary Setup)

- 1. Download the LyricSense application files.**
- 2. Import the provided database file into the UVA CS MySQL server using phpMyAdmin.**
- 3. Rename the database using your computing ID as required.**
- 4. Open `connect-db.php` and update:**
 - **database name**
 - **username**
 - **password**
- 5. Connect to the UVA network using Cisco Secure Client VPN.**

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

6. In the terminal, navigate to the project directory and run:

```
php -S localhost:8000
```

7. Open a browser and go to:

<http://localhost:8000>

The LyricSense application should load.

Option B: Local Setup (XAMPP)

1. Install XAMPP and start Apache and MySQL.
2. Open phpMyAdmin (<http://localhost/phpmyadmin>).
3. Create a new database (e.g., **lyricsense_db**).
4. Import the provided SQL file into the database.
5. Update **connect-db.php**:

```
$host = "localhost";  
$dbname = "lyricsense_db";  
$username = "root";  
$password = "";
```

6. Run:

```
php -S localhost:8000
```

7. Open:

<http://localhost:8000>

Option C: Google Cloud SQL (Cloud Deployment)

1. Create a MySQL instance in Google Cloud SQL.
2. Create a database and user credentials.
3. Import the project database using Cloud SQL import tools.

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

4. Update `connect-db.php`:

```
$host = "YOUR_CLOUD_SQL_IP";  
$dbname = "lyricsense_db";  
$username = "app_user";  
$password = "your_password";
```

5. Ensure your IP is authorized in Google Cloud SQL settings.

6. Run the application locally:

```
php -S localhost:8000
```

7. Navigate to:

<http://localhost:8000>

2.4 Advanced SQL Commands

1. Trigger

```
DELIMITER $$
```

```
CREATE TRIGGER update_lyrics_timestamp  
BEFORE UPDATE ON Lyrics  
FOR EACH ROW  
BEGIN  
    SET NEW.last_updated = NOW();  
END  
$$
```

```
DELIMITER ;
```

The trigger `update_lyrics_timestamp` was implemented on the `Lyrics` table to automatically update the `last_updated` attribute whenever a lyrics record is modified. This supports the application's edit feature, where users can update song details and lyrics through `update_song.php`.

By placing this logic directly in the database, the system ensures that modification timestamps are handled consistently without requiring extra PHP code. The trigger was tested by updating a

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

record in the **Lyrics** table. The lyrics content remained unchanged, but the **last_updated** timestamp automatically updated, confirming that the trigger executed correctly.

```
mysql>
mysql> DELIMITER ;
mysql> SELECT song_id, last_updated FROM Lyrics WHERE song_id = 1;
+-----+-----+
| song_id | last_updated |
+-----+-----+
|      1 | 2026-03-17 16:15:24 |
+-----+-----+
1 row in set (0.04 sec)

mysql> UPDATE Lyrics
  -> SET lyrics_text = lyrics_text
  -> WHERE song_id = 1;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT song_id, last_updated FROM Lyrics WHERE song_id = 1;
+-----+-----+
| song_id | last_updated |
+-----+-----+
|      1 | 2026-04-26 09:33:33 |
+-----+-----+
1 row in set (0.02 sec)

mysql>
```

2. Stored Procedure

DELIMITER \$\$

CREATE PROCEDURE GetSongsByMood(IN moodname VARCHAR(50))

BEGIN

SELECT s.song_title
FROM Songs s
JOIN TaggedWithMoods t ON s.song_id = t.song_id
JOIN Moods m ON t.mood_id = m.mood_id
WHERE m.mood_name = moodname;

END

\$\$

DELIMITER ;

The stored procedure **GetSongsByMood** was created to support mood based song retrieval. It accepts a mood name as input and joins the **Songs**, **TaggedWithMoods**, and **Moods** tables to return matching song titles. This reflects the relational design of the database because moods and songs are stored in separate tables and connected through a linking table, allowing a many to many relationship between songs and moods.

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

This procedure improves organization by centralizing the mood search logic inside the database instead of repeating the same query throughout the application. It was tested using a command such as:

```
CALL GetSongsByMood('sad');
```

The procedure successfully returned matching song records, confirming that the advanced SQL command works correctly and can support the application's mood search feature.

```
| I Still Have That Other Girl  
| Sleptalk Reimagined  
| Louder Now  
| Mommasan  
| Walk Out  
| Looking for a Sacrifice  
| Winter Hands  
| On the Cross Save Yourself Mix  
| This Is My E.L.F. Weapon  
| Struggle  
| Saturn  
+-----+  
3461 rows in set (0.06 sec)
```

3. Database Security - Database Level

3.1 Access Control

Database level access control was implemented by granting database privileges only to authorized members of the development team. Since the database is hosted on the UVA MySQL server, the project owner was able to manage access for collaborators, but server restrictions prevented the creation of a separate limited privilege application user. Because of this, developer access was handled at the database level, while end user restrictions were enforced through the application layer.

Only approved developers were permitted to modify the database, allowing the team to collaborate on schema updates, data management, and testing. This helped ensure that database changes were limited to trusted users while the application controlled what general users could do through the interface.

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

3.2 SQL Commands to Limit/Set Privileges

The following SQL commands were used to grant full database access to authorized developers:

```
GRANT ALL PRIVILEGES ON huz2ph_d.* TO 'rup5zv'@'%';
```

```
GRANT ALL PRIVILEGES ON huz2ph_d.* TO 'bwg4mg'@'%';
```

These commands allow the listed developers to view, insert, update, delete, and modify database objects within the `huz2ph_d` database. An additional restricted application user was considered, but the UVA MySQL server did not provide the required `CREATE USER` privilege. Therefore, limited end user access was implemented through application level protections instead.

4. Database Security - Application Level

4.1 Database Security on Application Level

Due to restrictions on the UVA MySQL server, it was not possible to create separate database users with limited privileges for application access. As a result, database security was enforced at the application level to control how users interact with the system and prevent unauthorized operations.

All interactions between the application and the database are mediated through PHP, which acts as a controlled interface. Users do not directly execute SQL queries. Instead, all requests are processed through predefined application logic that validates inputs and determines allowable actions. This ensures that only intended operations can be performed.

Prepared statements are used for all database queries to prevent SQL injection attacks by separating user input from SQL execution. Additionally, input validation is applied to ensure that only properly formatted and expected data is processed. Sensitive operations, such as deleting records, are restricted to POST requests and require verification through CSRF tokens. This prevents unauthorized actions that could otherwise be triggered through direct URL access or forged requests.

By enforcing these controls at the application level, the system ensures that database operations are secure, controlled, and resilient against common web vulnerabilities, even in the absence of fine grained database level user permissions.

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

4.2 Application Level DB Security Code Snippets

To enhance database security, our application enforces safeguards at the application level using PHP. These measures include the use of prepared statements, input validation, and Cross Site Request Forgery (CSRF) protection. Together, these mechanisms ensure that all database interactions are controlled, authenticated, and resistant to common web based attacks.

Prepared statements are used for all database queries to prevent SQL injection. By separating user input from SQL logic, the database engine treats input strictly as data rather than executable code. Additionally, user inputs are validated and sanitized before being processed.

For sensitive operations such as deleting records, we restrict execution to POST requests and require a valid CSRF token. This prevents malicious users from triggering actions through direct URL access or forged requests.

Example: Secure deletion using prepared statements and POST validation

```
verify_csrf_token();

if (!isset($_POST["song_id"])) {
    die("Invalid request.");
}

$song_id = (int) $_POST["song_id"];

$stmt = $pdo->prepare("DELETE FROM Songs WHERE song_id = :song_id");
$stmt->execute([
    ":song_id" => $song_id
]);
```

This implementation ensures that deletion requests must originate from the application interface and include valid, verified data.

Example: CSRF Token Generation

```
function csrf_token() {
    if (empty($_SESSION["csrf_token"])) {
        $_SESSION["csrf_token"] = bin2hex(random_bytes(32));
    }
    return $_SESSION["csrf_token"];
}
```

Course: CS4750: Database Systems

Project Title: LyricSense

Team Members: Trinity Chamblin (huz2ph), Victoria Lovelace (bwg4mg), Adeline Kwakye (rup5zv)

}

Example: CSRF Token Verification

```
function verify_csrf_token() {  
    if ($_SERVER["REQUEST_METHOD"] !== "POST") {  
        die("Invalid request method.");  
    }  
  
    if (  
        empty($_POST["csrf_token"]) ||  
        !hash_equals($_SESSION["csrf_token"], $_POST["csrf_token"])  
    ) {  
        die("Security validation failed.");  
    }  
}
```

These security mechanisms ensure that only legitimate, user-initiated actions are processed by the application. By combining prepared statements with CSRF protection and request validation, the system effectively prevents SQL injection, unauthorized data modification, and forged request attacks.